



泛型引入的技术背景

北京理工大学计算机学院
金旭亮

为什么要引入泛型 (1)

- JDK 1.5 之前的Java没有泛型，因此，JDK中的ArrayList等集合类型，其元素类型为Object，依据多态特性，这意味着可以向集合中加入任何一种类型的对象。

```
public class ArrayList {  
    public Object get( int i ) { . . . }  
    public void add( Object o ) { . . . }  
    . . .  
    private Object[] elementData;  
}
```

早期Java代码无法防止出现的编码错误

请看以下示例代码：

```
//创建一个只想保存字符串的List集合
List strList = new ArrayList();
strList.add("One string");
strList.add("Two string");
strList.add("Three string");

// “不小心” 把一个Integer对象 “丢进” 了集合
strList.add(5);
```

ListErr.java

- 当程序运行时，如果需要访问第4个元素并把它当成String，将引发ClassCastException。
- 这个错误必须等到“运行时”才能发现。

为什么要引入泛型 (2)

- 使用泛型，可以限制集合能接收的数据类型，并且消除了类型转换的需要。
- 新的泛型ArrayList定义了下面这样子的方法：

```
public class ArrayList<E>{  
    public E get(int index) {}  
    public boolean add(E e)  
    .....  
    private transient Object[] elementData;  
}
```



放在一对尖括号之内的符号称为“**类型参数**”，代表某种具体的数据类型。

GenericList.java示例剖析

使用泛型集合ArrayList<String>，可以在编译时就发现以下第2句代码有错误，但这种检查**仅发生于编译**时，在运行时不会进行这种检查工作：

```
//设定ArrayList中的数据全部都是String类型
List<String> strList = new ArrayList<String>();
//编译器不会允许这句代码通过编译
strList.add(5);
```

我们可以使用反射这种“无赖”手段绕开编译器的语法检查，直接将“5”加入集合而不会引发编译错误：

```
//使用反射技术，以下代码可以通过编译，但在运行时会由于“5”不是字符串，
//而引发一个ClassCastException
strList.getClass()
    .getMethod("add", Object.class)
    .invoke(strList, 5);
```

Java“泛型”相关术语



- 以ArrayList<E>、ArrayList<Integer>为例
 1. ArrayList<E>定义了一个泛型类型，“E”称为“**类型变量**”或“**类型参数**”。
 2. ArrayList<Integer>称为“**参数化的类型**”，“Integer”称为“**实际类型参数**”。
 3. ArrayList称为泛型类型ArrayList<E>的“**原始类型(raw type)**”。



注意区分泛型编程中的“raw type”与Java语言中的“primitive type”类型，primitive类型（int，float等）是与引用类型（reference type）相对应的概念。

泛型对象的“自动装箱”特性

```
ArrayList<Integer> list;  
list = new ArrayList<Integer>();  
// 将一个int数值加入到集合中  
list.add(3);
```

实际上是按以下这句编译的：

```
list.add( Integer.valueOf(3) );
```



Java中的集合，只能接收属于引用类型的对象，因此，上述代码需
要将int数值包进一个Integer对象中，这个特性称为“**装箱**
(boxing)”。

装箱特性实际上是由编译器完成的，它直接生成了相关字节码指令，
写入到了.class文件中

泛型对象的“自动拆箱”特性

- 以下语句：

```
int num = list.get(0);
```

- 将会被自动变换为：

```
int num = list.get(0).intValue();
```



尽可能明确地定义变量数据类型，尽量避免在大循环中出现装箱与拆箱现象。

了解泛型所带来的好处



1. 减少类的数目，促进代码复用
2. 初步剥离数据结构与算法
3. 减少编码错误：可以在编译时检查类型



在JDK中，有大量的组件（特别是集合类型）使用了泛型特性，在开发中，如果某组件有泛型和非泛型两个版本，则应该优先选择泛型版本。

Java 7的新特性

Java 7对泛型语法做了一些小的改动。

原先:

```
ArrayList<String> strs = new ArrayList<String>();
```

现在:

```
ArrayList<String> strs = new ArrayList<>();
```

减少了一些重复键入的工作。

下一讲



介绍Java中泛型类型与泛型方法的编写方法。